
Product Recognition on Store Shelves project report

Step A-B

Marco Scaramuzzi

25/09/2025

Contents

1.1	Project description	2
1.2	Step A	2
1.2.1	Task Introduction	2
1.2.2	Task overview	2
1.2.3	Methodology and Implementation Details	3
1.2.3.1	High-level pipeline	3
1.2.3.2	SIFT on multiple channels	4
1.2.3.3	Main implementation components	4
1.2.4	Results	6
1.2.5	Step A task conclusions	9

List of Figures

1	Workflow representation	3
2	Scene e1 bounding boxes	6
3	Scene e2 bounding boxes	6
4	Scene e3 bounding boxes	7
5	Scene e4 bounding boxes	8
6	Scene e5 bounding boxes	9

List of Tables

1.1 Project description

1.2 Step A

1.2.1 Task Introduction

The task A requires to implement a single-instance product detector that localizes cereal boxes in shelf images.

The task was attempted using SIFT features, computed independently on the RGB channels, FLANN matching with Lowe's ratio test, and homography estimation with RANSAC. The notebook [step_A.ipynb](#) contains the runnable pipeline; this report highlights the workflow, key data structures, concise results (extracted from the notebook output), and a short conclusion. For Step A, all models were correctly recognized.

1.2.2 Task overview

The project consists of a pipeline for detecting products on store shelves using computer vision techniques. The main steps include loading images, extracting features, matching them, and estimating the position of products in the scene.

The processing pipeline is compactly represented below.

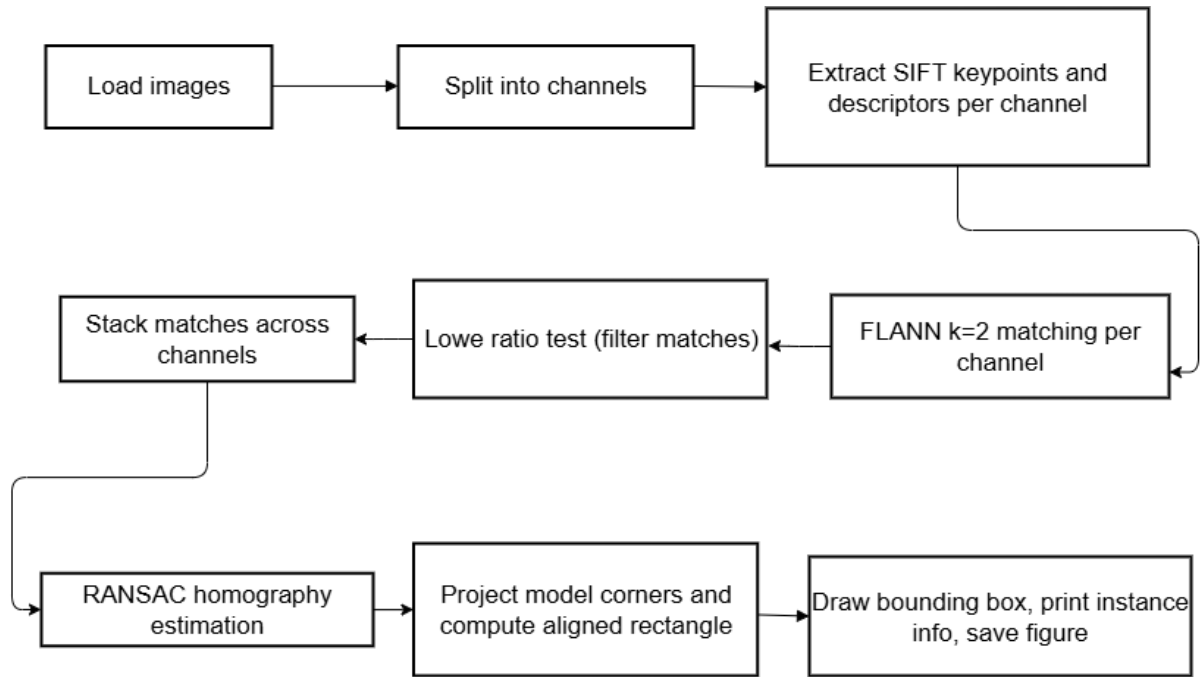


Figure 1: Workflow representation

1.2.3 Methodology and Implementation Details

This section gives a deeper explanation of the pipeline (both theoretical motivations and implementation details) and the main in-memory data structures used by the notebook.

1.2.3.1 High-level pipeline

1. **Per-channel feature extraction:** Each model and scene image is split into R/G/B channels (function `create_channel_dict`) and SIFT keypoints/descriptors are computed independently on each channel using `cv2.SIFT_create()` inside `extract_features_dict`. The result is two nested dictionaries (`keypoints_dict` and `descriptors_dict`) keyed by image id and channel.
2. **Channel-wise matching:** For each model-vs-scene pair, descriptors from channel `c` of the model are matched against descriptors from channel `c` of the scene using a FLANN approximate k-NN search (`k=2`). The reason for `k=2` is to enable Lowe's ratio test which requires the nearest and second-nearest neighbors.
3. **Ratio-based filtering:** The `filter_matches` function applies Lowe's ratio test, which is expressed as the formula $\text{dist}_1 < 0.7 \times \text{dist}_2$. Here, dist_1 represents the distance to the nearest neighbor, and dist_2 represents the distance to the second-nearest neighbor in the descriptor

space. This test removes ambiguous matches where the nearest neighbor is not sufficiently better than the second nearest.

4. **Aggregation and geometric verification:** The filtered ‘good’ matches from the three channels are stacked into arrays of source and destination points. These stacked correspondences feed `cv2.findHomography(..., cv2.RANSAC, ransacReprojThreshold=3)`. RANSAC rejects outlier matches and returns a robust 3×3 homography matrix that maps model coordinates to the scene.
5. **Projection and bounding rectangle:** Model corners are projected into the scene with `cv2.perspectiveTransform`. To produce a stable, axis-aligned bounding box, the notebook calls `compute_aligned_rectangle` (in `utils/bounding_box_utils.py`), which centers and constrains the rectangle to image bounds and computes width, height, and center.

1.2.3.2 SIFT on multiple channels

- **Color-preserving detection:** converting to grayscale discards chromatic edges and contrast that can be crucial for brand logos or colored patterns. Computing SIFT independently on R/G/B preserves such color-specific details.
- **Complementary matches:** features that are weak in one channel may be strong in another; stacking channel-wise good matches increases the chance of obtaining enough inliers for RANSAC while still allowing geometric pruning of false matches.
- **Practical trade-offs:** processing three channels multiplies descriptor storage and matching work by $\approx \times 3$. The notebook mitigates false positives by applying the Lowe ratio test per-channel and then applying RANSAC on stacked matches.

1.2.3.3 Main implementation components

- SIFT: instantiated via `sift = cv2.SIFT_create()` and applied per-channel in `extract_features_dict`. Model images were resized to a common shape (180 × 240) before feature extraction to reduce inter-model scale variability. This normalization stabilizes keypoint distribution and simplifies numeric thresholds like `min_count`.
- FLANN: the matcher is built in `initialize_flann()` with KD-tree (`trees=5`) and search `checks=50` (these parameters are tuned for a balance between speed and accuracy). FLANN’s `knnMatch(..., k=2)` returns pairs used by Lowe’s ratio. It is particularly efficient for high-dimensional descriptor matching, leveraging approximate nearest neighbor search to balance speed and accuracy.
- Lowe ratio test: implemented in `filter_matches` with threshold 0.7 (a standard value; higher values are stricter). The function returns only the best matches per channel.

- RANSAC/Homography: `compute_homography` stacks matched keypoint coordinates and calls `cv2.findHomography(..., cv2.RANSAC, ransacReprojThreshold=3)` to estimate a robust transformation. The reprojection threshold (3 px) controls inlier acceptance.

Listing 1: Representative In-Memory Structures

```

1 keypoints_dict = {
2     model_indexes[0]: {
3         'R': [<cv2.KeyPoint 0x1>, <cv2.KeyPoint 0x2>],
4         'G': [<cv2.KeyPoint 0x3>, <cv2.KeyPoint 0x4>],
5         'B': [<cv2.KeyPoint 0x5>, <cv2.KeyPoint 0x6>]
6     },
7     model_indexes[1]: {
8         'R': [<cv2.KeyPoint 0x7>, <cv2.KeyPoint 0x8>],
9         'G': [<cv2.KeyPoint 0x9>, <cv2.KeyPoint 0x10>],
10        'B': [<cv2.KeyPoint 0x11>, <cv2.KeyPoint 0x12>]
11    }
12    # ... more keys for each image id
13 }
14
15 # structure of descriptors_dict is similar to keypoints_dict, the inner
    dictionaries values are numpy arrays instead of list of keypoints

```



model images were resized to a common shape (180x240) before feature extraction to reduce inter-model scale variability. Although SIFT is scale-invariant theoretically, in practical template matching scenarios large differences in template sizes lead to inconsistent keypoint counts, differing keypoint scale distributions, and mismatched descriptor sampling densities. Normalizing templates to a common scale stabilizes the number and distribution of keypoints per model, simplifies the choice of numeric thresholds (e.g. `min_count`), and reduces false negatives caused by extreme template-to-scene scale mismatches. The trade-off is that very large scale variations in the scene may require multi-scale search or pyramid matching, but for a controlled, single-instance template setup this normalization improves consistency and reproducibility.

Suggested figures, tables and additional content to add for rigor

- Per-channel keypoint panels: for a representative model and scene, show three small images with SIFT keypoints plotted on R, G, and B channels respectively.
- Match-count table: CSV-style table with rows for (scene, model) and columns `matches_R`, `matches_G`, `matches_B`, `total_matches`. This empirically supports the `min_count` choice.

1.2.4 Results

Below are the detections produced by `main(min_count=75, ...)` for the five scenes. The provided images represent the bounding boxes of the detected models drawn on each scene image and the textual output lists the detected products instances alongside the width/height and position of the center of the bounding box.

- Scene e1 — visualization



Figure 2: Scene e1 bounding boxes

- Product 0 — 1 instance found:

- * Instance 1 {position: (162, 215), width: 309px, height: 430px}

- Product 11 — 1 instance found:

- * Instance 1 {position: (444, 180), width: 299px, height: 361px}

- Scene e2 — visualization

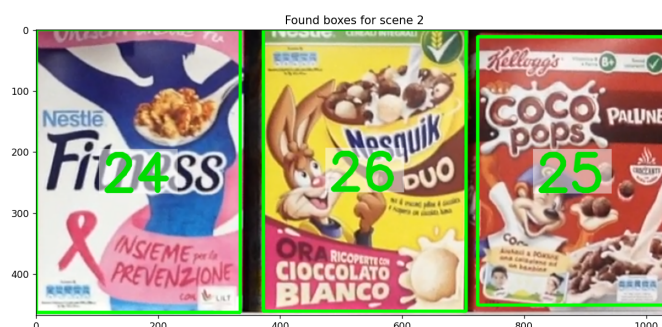


Figure 3: Scene e2 bounding boxes

- Product 24 — 1 instance found:
 - * Instance 1 {position: (167, 232), width: 334px, height: 464px}
 - Product 25 — 1 instance found:
 - * Instance 1 {position: (878, 232), width: 312px, height: 440px}
 - Product 26 — 1 instance found:
 - * Instance 1 {position: (538, 230), width: 333px, height: 461px}
- Scene e3 — visualization



Figure 4: Scene e3 bounding boxes

- Product 0 — 1 instance found:
 - * Instance 1 {position: (170, 234), width: 323px, height: 435px}
 - Product 1 — 1 instance found:
 - * Instance 1 {position: (818, 198), width: 303px, height: 396px}
 - Product 11 — 1 instance found:
 - * Instance 1 {position: (476, 192), width: 303px, height: 385px}
- Scene e4 — visualization



Figure 5: Scene e4 bounding boxes

- Product 0 — 1 instance found:
 - * Instance 1 {position: (160, 738), width: 320px, height: 435px}
- Product 11 — 1 instance found:
 - * Instance 1 {position: (464, 688), width: 303px, height: 395px}
- Product 25 — 1 instance found:
 - * Instance 1 {position: (554, 218), width: 319px, height: 435px}
- Product 26 — 1 instance found:
 - * Instance 1 {position: (206, 221), width: 341px, height: 442px}
- Scene e5 — visualization



Figure 6: Scene e5 bounding boxes

- Product 19 — 1 instance found:
 - * Instance 1 {position: (504, 191), width: 295px, height: 382px}
- Product 25 — 1 instance found:
 - * Instance 1 {position: (161, 228), width: 320px, height: 444px}

1.2.5 Step A task conclusions

The SIFT-per-channel + FLANN + RANSAC pipeline reliably localizes single instances of the provided product models in the five test scenes. The saved [step_a/figures/scene_*.png](#) images contain the annotated outputs used to verify detections.